

Dockerized-Microservices — The Lesson

Chain C · Project C-2 · the full written lesson, start to finish — compiled from docs/interactive/index.html

1 — Why Docker? The Works-On-My-Machine Problem

C-1's app assumes a Python version, a set of installed packages, and a running Redis — all implicit, all living only in your head and your laptop's state. A Dockerfile makes every one of those assumptions explicit and reproducible: `FROM python:3.12-slim` pins the interpreter, `COPY requirements.txt + pip install` pins the dependencies, and the resulting image runs identically on a teammate's machine, in CI, and in production.

Docker builds an image in **layers**, one per instruction, and caches each layer. A later build only re-runs the instructions from the first changed layer onward — everything before that is reused instantly from cache.

The cost of getting the order wrong: if application code is copied before dependencies are installed, editing a single line of code invalidates every layer after it — including the slow `pip install` step. Correct ordering means a code-only change rebuilds in seconds; reversed, it rebuilds in minutes, every time.

```
# api/Dockerfile — order matters here
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /install /usr/local
COPY app/ app/
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

2 — Docker Compose: Wiring Services Together

Four services, one `docker-compose.yml`: `redis` (broker), `api` and `worker` (the **same built image**, different `command`), and `nginx` (public door). Service discovery is by name — `api` can reach `redis://redis:6379/0` because Docker's embedded DNS resolves the service name `redis` to that container's current IP, no hardcoding, no lookup table to keep in sync.

The trap — and a real bug this project hit: the first working version of this stack started cleanly, connected to Redis, and accepted applications through the API with no errors — but the worker never actually processed anything. The root cause: the Celery app was constructed without `include=["app.tasks"]`. The worker process starts from `app.celery_app`, which never imports `app.main` or `app.tasks` on its own — so `@celery_app.task` in `tasks.py` never ran on the worker, the task

name was never registered, and the enqueued work sat in Redis forever, unclaimed. Celery does not error on this — from the broker's perspective, the message is perfectly valid. Fixed by passing `include=["app.tasks"]` to the `Celery()` constructor.

```
# docker-compose.yml (excerpt)
worker:
  build: ./api
  command: celery -A app.celery_app worker --loglevel=info
  depends_on:
    redis:
      condition: service_healthy
```

3 — Nginx as Reverse Proxy

Only `nginx` publishes a port to the host (`8080:80`) — `api`'s `8000` and `redis`'s `6379` are reachable only from inside the Docker network. Nginx's `location` blocks route by path: `/api/` proxies to the FastAPI upstream, everything else serves static files.

The trap: without `try_files $uri /index.html`, refreshing the browser on a client-side route (like `/jobs/3` in a React Router app) 404s against Nginx's filesystem before the frontend's own router ever gets a chance to run — because that path isn't a real file on disk. The fallback is what hands control back to the SPA.

```
# nginx/nginx.conf (excerpt)
upstream api_upstream { server api:8000; }

location /api/ {
  proxy_pass http://api_upstream;
}
location / {
  root /usr/share/nginx/html;
  try_files $uri /index.html;
}
```

4 — CI/CD: Test, Then Build, Then Push

Two jobs, not one. `test` runs `pytest` against the API. `build-and-push` has `needs: test` — GitHub Actions will not even start it until `test` finishes successfully. Only then does it log into GHCR (using the auto-provided `secrets.GITHUB_TOKEN`, no manual credential setup) and push the image tagged by the commit's git SHA.

Why two jobs instead of one: if "run tests" and "build + push" were the same job, there would be no hard gate — a build step could succeed even against code that failed its own tests, if the steps weren't ordered carefully. Splitting into two jobs with `needs:` makes "tests passed" a real, independent

precondition that GitHub Actions enforces structurally, not just a step ordering you have to get right by hand.

```
# .github/workflows/ci.yml (excerpt)
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - run: pytest api/tests/ -v

  build-and-push:
    needs: test
    if: github.event_name == 'push'
    steps:
      - uses: docker/build-push-action@v6
        with:
          tags: ghcr.io/${{ github.repository }}:${{ github.sha }}
```